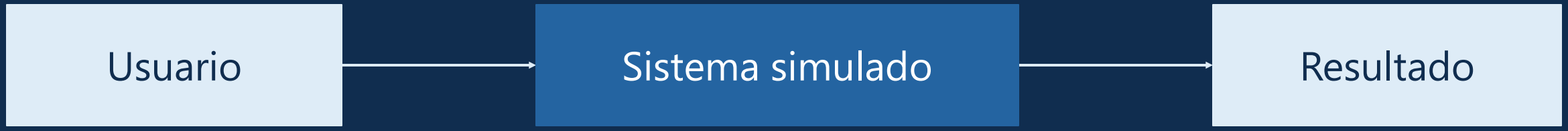
Simulador de interfaz de llamadas al sistema

Proyecto de Arquitectura de Sistemas Operativos



Cris, Jean y Alejandra

Python, Tkinter y ttk

Problema que aborda el proyecto

Una llamada al sistema

Es el mecanismo con el que un programa de usuario solicita un servicio al núcleo del sistema operativo.



El estudiante ve la acción y la respuesta.
El simulador permite explicar las capas intermedias.

Objetivo y alcance

El simulador sí representa

Solicitudes y respuestas

Procesos simulados en memoria

Archivos físicos controlados

Validaciones y errores

Bitácora de cada solicitud

El simulador no implementa

Kernel real

Procesos de Windows

Cambio real de privilegios

Scheduler

Drivers o interrupciones

Propósito: comprender la comunicación entre usuario y kernel

Modo usuario y kernel simulado

Modo usuario conceptual

GUI recoge entradas

Aplicación solicita el servicio

Interfaz estructura la solicitud

Kernel simulado

Dispatcher valida y enruta

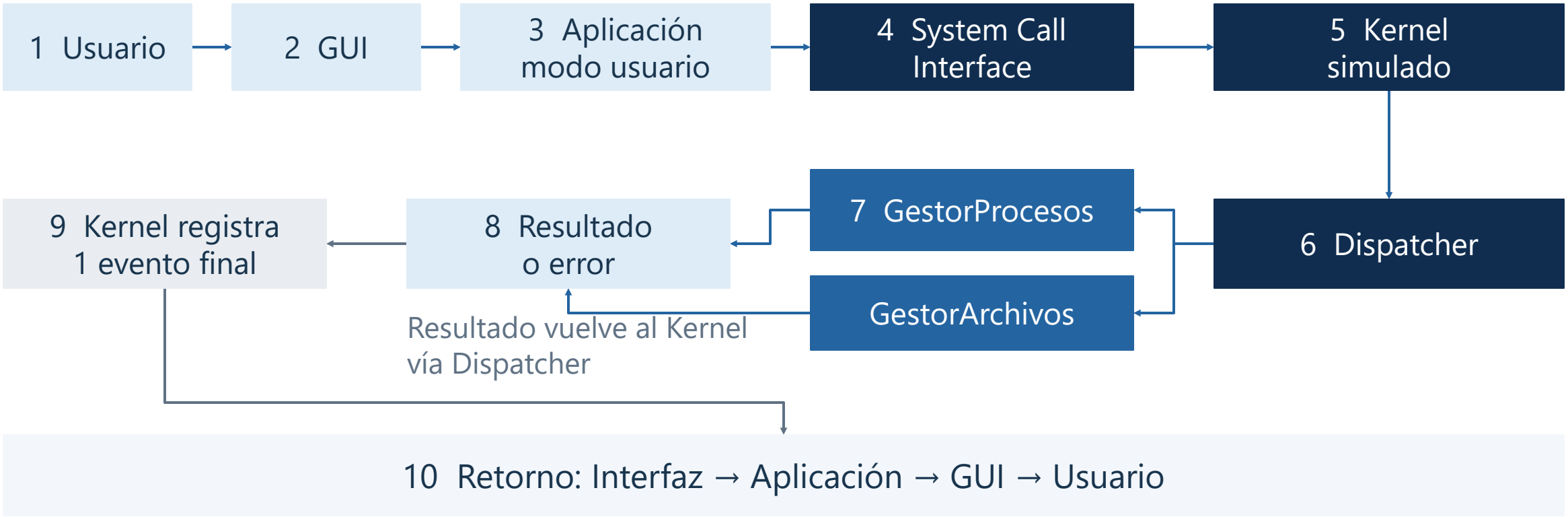
Gestor valida el recurso

Kernel registra y retorna



La transición es conceptual. Python no cambia los privilegios del procesador.

Arquitectura del simulador



Gestores: procesos en memoria o archivos bajo datos/archivos/

Las 11 llamadas disponibles

Procesos: 4 llamadas

PROC-01	create_process	Crear
PROC-02	list_processes	Listar
PROC-03	get_process	Consultar
PROC-04	terminate_process	Finalizar

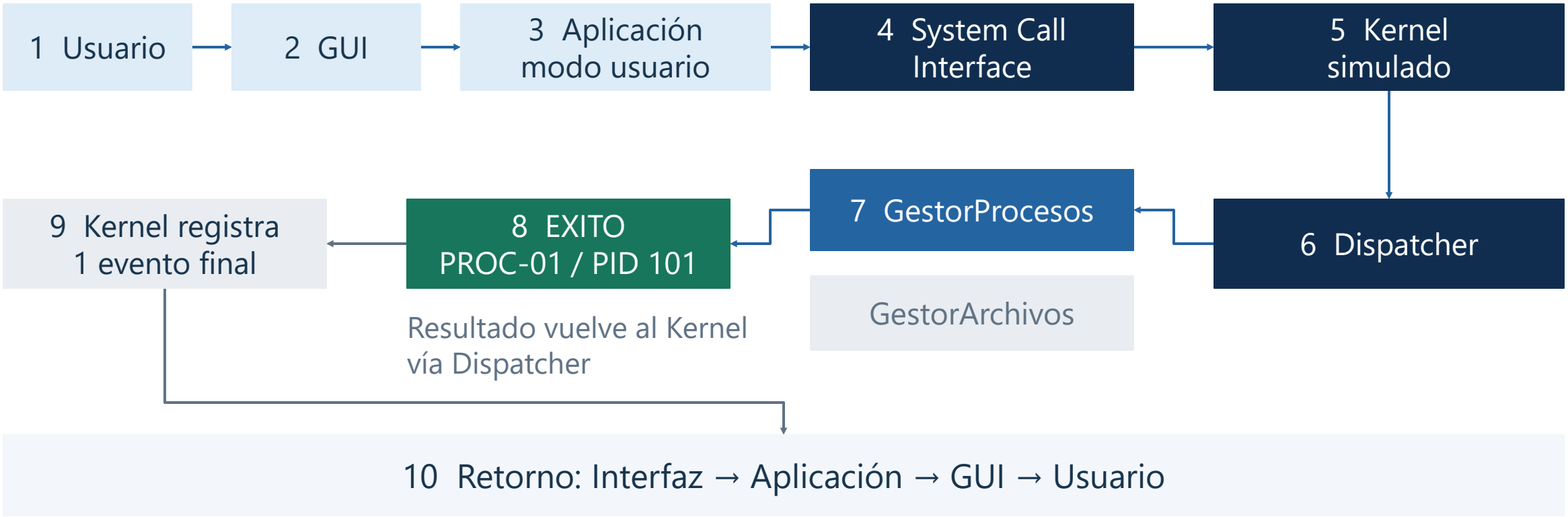
Archivos: 7 llamadas

FILE-01	create_file	Crear
FILE-02	open_file	Abrir
FILE-03	read_file	Leer
FILE-04	write_file	Escribir
FILE-05	close_file	Cerrar
FILE-06	delete_file	Eliminar
FILE-07	list_files	Listar

Todas devuelven estado, codigo, mensaje y datos

Recorrido de create_process

create_process("Editor", 3)

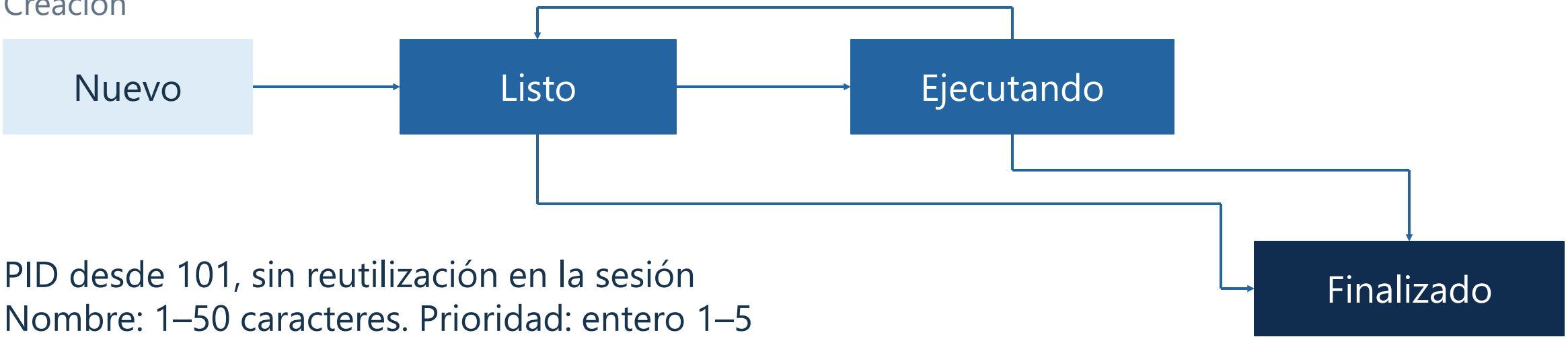


Dentro del gestor: validar → PID 101 → Nuevo → Listo

Modelo y estados de procesos

PID	Nombre	Estado	Prioridad
101	Editor	Listo	3

Modelo interno: máximo un Ejecutando
Creación



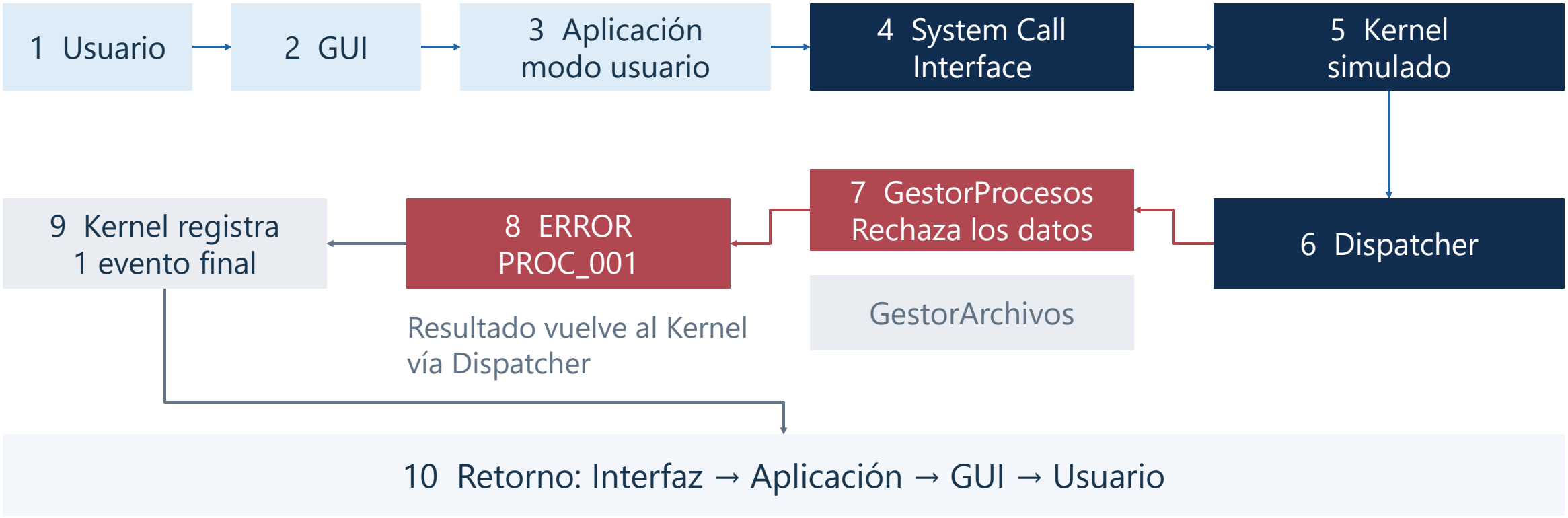
PID desde 101, sin reutilización en la sesión
Nombre: 1–50 caracteres. Prioridad: entero 1–5

Prioridad informativa. Sin scheduler ni llamada pública para Ejecutando

Recorrido de una solicitud inválida

create_process("", 6)

No crea proceso. No consume PID.

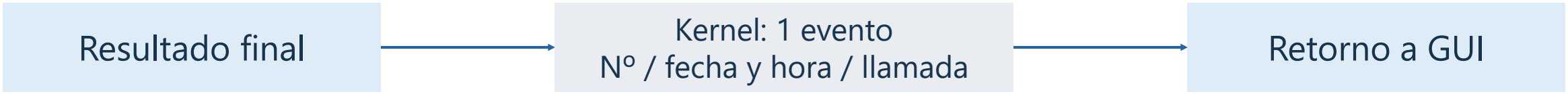


Estructura: SYS_001 / SYS_002 en Dispatcher Dominio: PROC_001–003 en gestor

Resultados y bitácora

Resultado uniforme

Campo	Éxito	Error
estado	EXITO	ERROR
codigo	PROC-01	PROC_002
mensaje	Proceso creado correctamente.	PID inválido o inexistente.
datos	{"pid": 101}	{}



Parámetros y resultado. Solo lectura. Registra cantidades, sin copiar contenido.

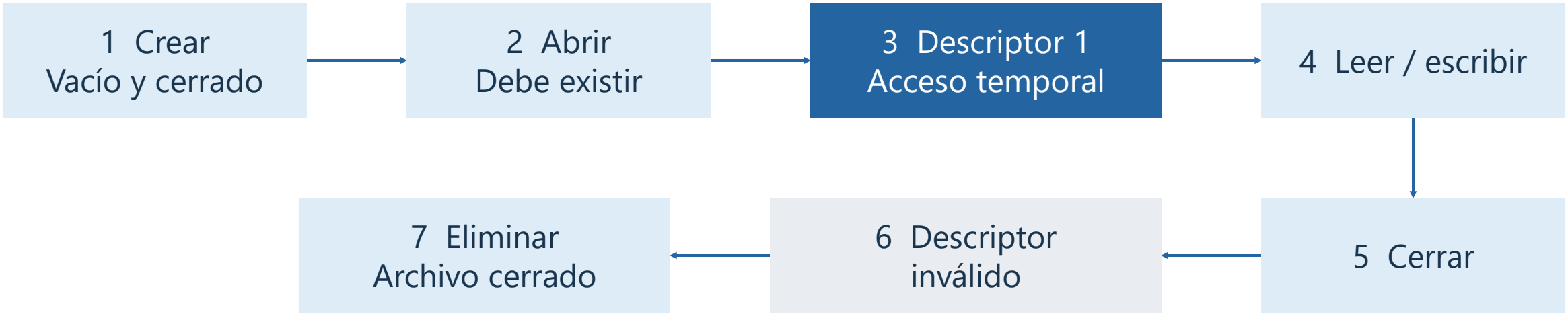
Archivos físicos y descriptores simulados

Físico

datos/archivos/ejemplo.txt
El contenido sobrevive entre sesiones

Simulado

Descriptor 1
Identifica una apertura de esta sesión



Descriptores desde 1, sin reutilización en la sesión. No son manejadores de Windows.

Modos y permisos de archivo

Modo	Leer	Escribir
r	Sí	No
w	No	Sí
rw	Sí	Sí

Una sola apertura por archivo

Abrir en w no crea ni trunca

Escribir reemplaza todo

Cerrar invalida el descriptor

Eliminar exige archivo cerrado

Ejemplo: "Hola mundo" + write_file(1, "Hola") = "Hola"

El modo del simulador controla permisos conceptuales de esa apertura.

Confinamiento del almacenamiento

Zona autorizada

datos/archivos/

ejemplo.txt
Nombre simple permitido

Solicitudes rechazadas

../secreto.txt / \

C:\archivo.txt

\\servidor\recurso

CON, NUL, COM1, LPT1

archivo.txt:flujo

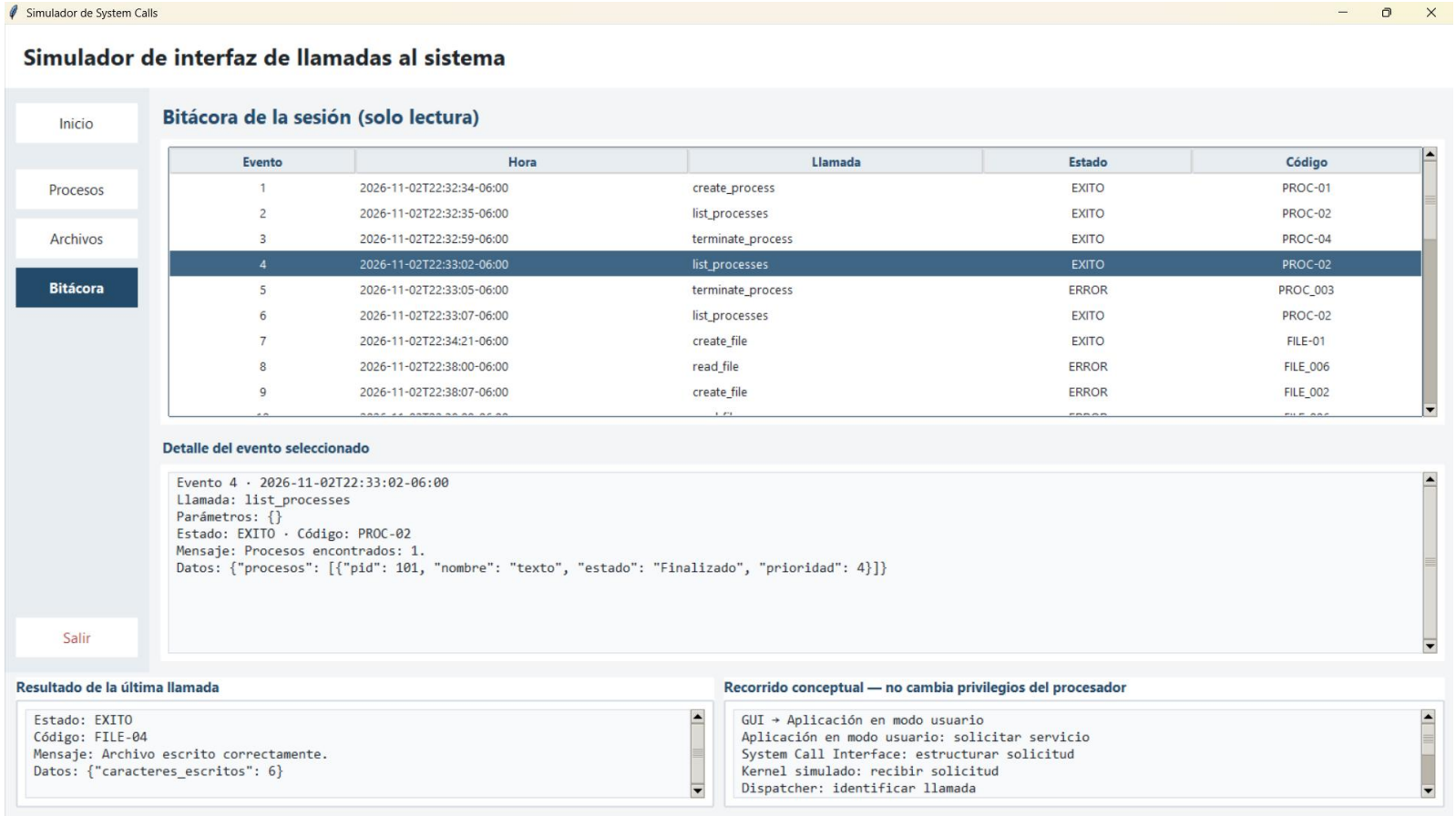
Symlinks, junctions y hard links

Revalida raíz y destino antes de leer o escribir.
list_files() no expone archivos externos.

No es una sandbox completa ni protege frente a cambios concurrentes externos.

Interfaz gráfica y demostración

Inicio, Procesos, Archivos y Bitácora



1 Crear y listar proceso

2 Consultar proceso

3 Crear y abrir archivo

4 Escribir y leer

5 Cerrar y listar

6 Provocar FILE_008

7 Consultar bitácora

8 Finalizar proceso

9 Cerrar y eliminar

Panel inferior izquierdo: resultado. Panel inferior derecho: recorrido conceptual.

Pruebas y resultados

58	56	2	0
casos en la suite documentada	correctas	omitidas	fallos

16 solicitudes = 16 eventos

ENTREGA VERIFICADA

2 symlinks omitidos por WinError 1314. Hard links: prueba ejecutada.
Tk abrió una ventana real. No sustituye una inspección visual manual completa.

Evidencia documentada: 11 de octubre de 2026

Conclusiones

Una solicitud, capas con responsabilidades claras y un resultado que puede comprobarse

Procesos simulados y archivos físicos confinados

Resultados uniformes para éxitos y errores

Un evento final por solicitud

Pruebas documentadas con límites explícitos

¿Preguntas?

Catálogo de errores

Procesos y solicitud

PROC_001	Datos de proceso inválidos
PROC_002	PID inválido o inexistente
PROC_003	Proceso ya finalizado
SYS_001	Llamada desconocida
SYS_002	Parámetros inválidos o incompletos

Archivos

FILE_001	Nombre o ruta inválida
FILE_002	Archivo ya existente
FILE_003	Archivo inexistente
FILE_004	Modo de apertura inválido
FILE_005	Apertura duplicada
FILE_006	Descriptor inválido o cerrado
FILE_007	Operación no permitida por modo
FILE_008	Eliminar archivo abierto

FILE_009: fallo físico. Decisión interna, fuera del catálogo académico.

Trazabilidad de requisitos y pruebas

Grupo	Qué comprueba	Módulos de pruebas
CP-01 a CP-04	Crear, listar, consultar y finalizar procesos	test_integracion.py test_procesos.py
CA-01 a CA-07	Siete operaciones de archivos	test_integracion.py test_archivos.py
CE-01 a CE-10	Datos, PID, archivos y solicitudes inválidas	test_integracion.py test_procesos.py test_archivos.py

GUI: test_gui_archivos.py
Integración: resultado y evento final por solicitud
Demostración pública: demo.py, 16 solicitudes

Decisiones y límites técnicos

Decisiones del equipo

Python en lugar de C

Tkinter y ttk para la GUI

Sin evidencia de autorización docente del cambio de lenguaje

Decisiones internas y límites

Ejecutando solo interno, sin scheduler

FILE_009: extensión interna pendiente de confirmación académica

Bitácora y procesos en memoria

Sin protección frente a concurrencia

2 pruebas de symlink omitidas

La propuesta especifica C para el tema 7. El documento técnico diseña Python.